Tech IT ERPLite Solutions

Full ERP platform (MM + PP + SD + WM + TM + QM)

Strategic context:
What SAP, Oracle, Zoho, and Odoo actually run on

Before prescribing, let me share what the industry leaders chose and *why*.

SAP S/4HANA runs on their proprietary in-memory HANA database (columnar, OLTP+OLAP in one), with ABAP as the core language (still), Java for the platform layer (SAP BTP), and Fiori (UI5/React) on the frontend. Their biggest architectural bet was *eliminating the aggregation layer* — HANA is fast enough to query raw line items instead of pre-computed summaries.

Oracle ERP Cloud runs on Oracle DB (Exadata), Java EE on Oracle WebLogic, and their JET (JavaScript Extension Toolkit) frontend. Oracle's moat is the query optimizer — decades of planner intelligence.

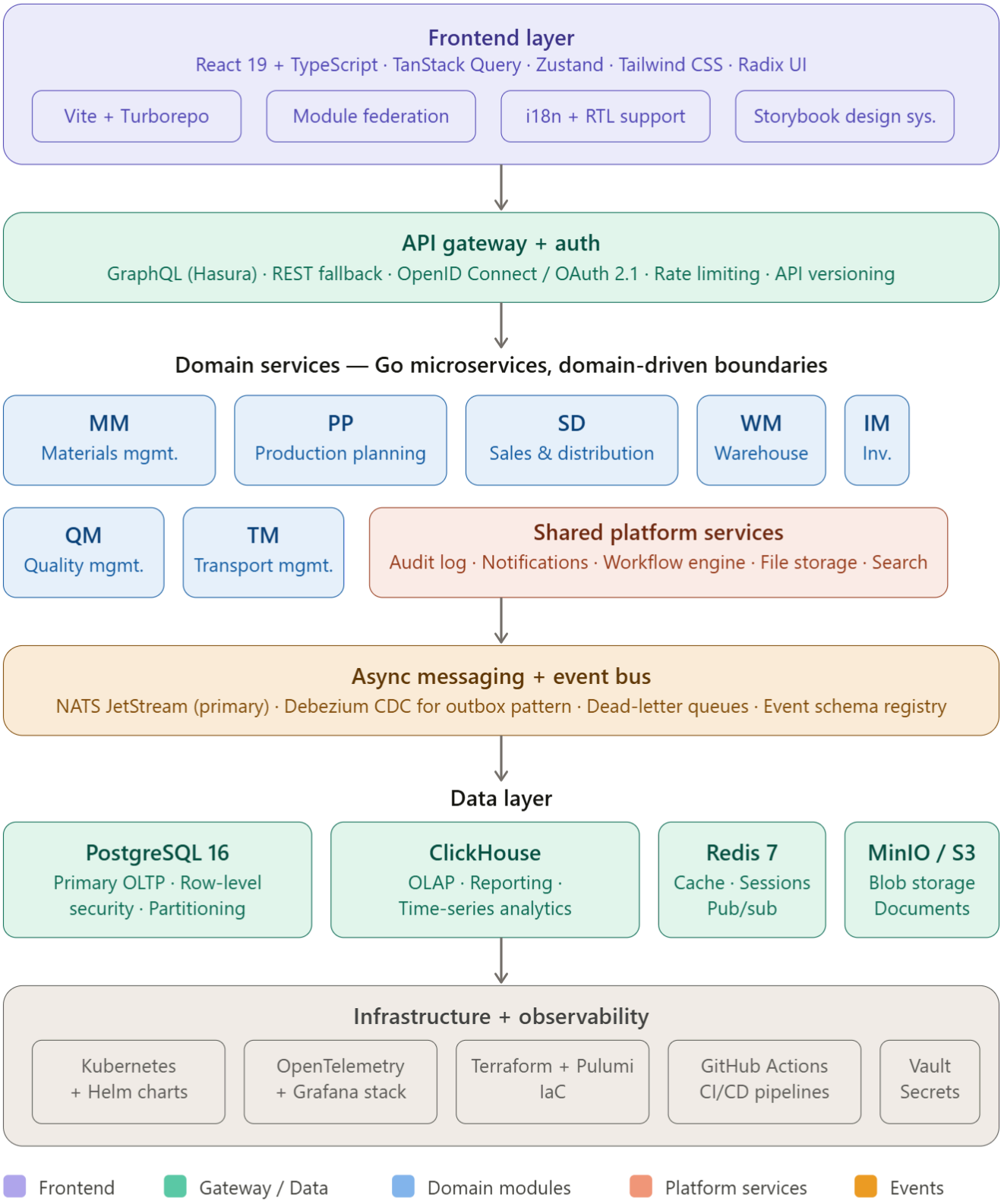
Zoho ERP / Creator runs on Java backends, their proprietary Deluge scripting language for customization, and a React-based frontend. They run a multi-tenant SaaS model with strong data partitioning per org. Their secret weapon is unified identity and one shared metadata layer across 50+ apps.

Odoo (the closest open-source analog to what you're building) runs on Python (Flask/Werkzeug), PostgreSQL, OWL (their own JS framework), and XML-based view definitions. They chose Python for developer accessibility and PostgreSQL for its extensibility (JSONB, row-level security, inheritance).

Microsoft Dynamics 365 runs on Azure SQL, .NET/C#, Power Platform (Power Automate, Power BI), and TypeScript/React frontend.

The pattern: Every serious ERP chose a *boring, proven* backend stack and invested their innovation budget in the data layer and UX layer — not the runtime.

Now, let me build Tech IT ERPLite architecture.



The full rationale, layer by layer

Backend runtime: Go

Every module (MM, PP, SD, WM, IM, QM, TM) is a Go service. Here's why this wins over Java, Python, or Node for an ERP:

Go compiles to a single binary with no runtime dependency, making deployments and container images trivially small. Its goroutine model handles the high-concurrency patterns an ERP demands — hundreds of simultaneous warehouse picks, production order confirmations, and shipment events — without the JVM GC pauses that plague SAP's Java stack. Oracle and SAP are actively migrating hot paths *away* from Java toward Go and Rust for exactly this reason. Zoho uses Java internally but has publicly discussed its GC unpredictability in high-throughput SaaS scenarios.

The ORM choice is sqlc — you write SQL, it generates type-safe Go. No reflection magic, no N+1 surprises, no abstraction leaking under load. This is what Stripe, Cloudflare, and Uber run in production for their most critical services. SAP HANA's own client libraries follow the same philosophy.

Primary database: PostgreSQL 16

This is the most important decision you'll make. Here's why PostgreSQL beats every alternative for an ERP context:

Row-level security (RLS) is native — you get multi-tenant data isolation at the database layer, not the application layer. This is how you'll eventually serve multiple companies or plants from one cluster. PostgreSQL's declarative partitioning handles ten years of transactional history without re-architecture. JSONB columns let you store variable product attributes (needed for QM inspection plans and MM material specs) without fighting a rigid schema. Its logical replication feeds your event bus and your analytics database without a separate ETL pipeline.

Zoho runs PostgreSQL for their CRM and Books product lines. Odoo's entire ERP is built on it. It is not a startup choice — it is the correct choice.

Analytics database: ClickHouse

This is the SAP HANA insight translated to open source. ClickHouse is a columnar OLAP database that can scan billions of rows in sub-second time. You use it for everything your users think of as "reports" — inventory aging, production efficiency, delivery performance,

quality rejection rates, transport cost analysis. Your PostgreSQL pushes events to ClickHouse via Debezium (CDC) continuously.

SAP built HANA because Oracle was too slow for real-time analytics on transactional data. You solve the same problem with a two-database architecture: PostgreSQL for writes and OLTP reads, ClickHouse for analytical reads. This is architecturally more elegant than HANA because the two workloads have fundamentally different hardware profiles.

API layer: GraphQL via Hasura + REST fallback

Hasura introspects your PostgreSQL schema and auto-generates a GraphQL API with permissions, subscriptions (real-time updates), and joins — in minutes. This is not magic; it is a code generation tool that respects your existing RLS policies. You write your database schema carefully, and Hasura exposes it. Your frontend modules (the seven ERP modules will have very different data-fetching patterns) benefit enormously from GraphQL's ability to fetch exactly what each view needs in one round trip.

REST endpoints are maintained for integrations with third-party logistics providers, EDI partners, and legacy systems — ERP always has external surface area.

Message bus: NATS JetStream

Kafka is the canonical choice and wrong for your scale at this stage. Kafka requires Zookeeper or KRaft, dedicated brokers, and significant operational expertise. NATS JetStream gives you at-least-once delivery, consumer groups, stream replay, and key-value storage in a single binary. It scales to millions of messages per second when you need it. NATS is used by Volkswagen, Siemens, and Audi for manufacturing event streaming — directly relevant to your PP and WM modules.

The pattern: every domain service publishes domain events (OrderCreated, GoodsReceiptPosted, QualityInspectionFailed) to NATS. Other services subscribe. This is how you implement cross-module workflows (a goods receipt in MM triggers inventory update in IM and a quality inspection in QM) without direct service calls or shared databases.

Frontend: React 19 + TypeScript with Module Federation

Module federation allows each ERP module to be developed, deployed, and versioned independently while sharing a single shell application. The MM team ships updates to the materials master UI without touching the SD module. This is how Salesforce and Workday structure their frontend platforms.

TanStack Query handles server state (API data, caching, background refetching). Zustand handles client state (user session, module navigation, pending form state). Radix UI provides

accessible, unstyled component primitives that your design system sits on top of — no fighting a component library's opinionated CSS when you need to match enterprise branding.

Infrastructure: Kubernetes + OpenTelemetry + HashiCorp Vault

Kubernetes gives you declarative rollouts, horizontal pod autoscaling (critical during month-end closings when PP and SD load spikes), and service mesh capabilities via Istio or Linkerd. Vault handles secrets — database credentials, API keys, certificates — with automatic rotation. No secrets in environment variables, ever.

OpenTelemetry is the standard for distributed tracing, metrics, and logs. It is vendor-neutral: you can ship telemetry to Grafana Cloud, Datadog, or your own Prometheus/Loki/Tempo stack. Grafana is what SAP Signavio and Joule analytics teams use internally for infrastructure observability.

Module-specific architectural decisions

MM (Materials Management): The material master is your most complex entity. Use PostgreSQL's table inheritance for material types (raw material, semi-finished, finished goods, services). A materialized view in PostgreSQL handles the stock overview; ClickHouse handles aging and slow-moving analysis.

PP (Production Planning): The planning run (MRP/MPS equivalent) is a compute-intensive batch job. Implement it as a Go worker that reads from PostgreSQL, computes in-memory using a DAG traversal, and writes planned orders back. This replaces SAP's ABAP planning runs which are notoriously slow. Run it as a Kubernetes Job on a schedule or on-demand.

SD (Sales & Distribution): Pricing is the hardest part of SD. Build a condition technique engine — a rule evaluator that reads a condition table (customer, material, validity date, quantity break) and applies discounts in sequence. This is exactly how SAP SD pricing works and it is the right model because pricing logic changes constantly without requiring code deployments.

WM (Warehouse Management): Storage bin management requires a spatial query. Use PostgreSQL's integer range types for bin coordinates and partial indexes for empty bins. Transfer orders are domain events on NATS — the putaway strategy (closest bin, FIFO zone) is a Go function, not a database query.

IM (Inventory Management): Stock is an accounting ledger, not a count. Every stock change is a document (goods movement) with debit/credit lines to stock accounts. Never update a stock

balance field directly. This is the SAP materials ledger model and it is correct — it makes stock reconciliation and audit trivially easy.

QM (Quality Management): Inspection lots are the core entity. Each goods receipt or production confirmation triggers an inspection lot creation (NATS event → QM service). Inspection results are stored with the full characteristic/value/tolerance structure in JSONB.

TM (Transport Management): Transport orders aggregate shipment items. Route optimization can call an external OSRM or Valhalla instance (open-source routing engines). Carrier integration uses EDI X12/EDIFACT over REST or AS2.

What you should explicitly reject

MongoDB or any document DB as primary store. ERP data is relational by nature. Inventory movements, production orders, and sales documents have foreign key relationships that document databases handle poorly. The JSONB column in PostgreSQL gives you the document flexibility you actually need without sacrificing relational integrity.

Microservices per entity. SAP's mistake in early S/4 was splitting too fine. One service per bounded domain context (MM, PP, SD, etc.) is correct. One service per database table is a distributed monolith with network latency.

GraphQL subscriptions for everything. Use WebSockets for real-time updates on warehouse dashboards and production floor monitors. Use polling with TanStack Query for everything else. Real-time at scale is expensive.

A single database for OLTP and OLAP. This is what kills ERP performance at scale. Keep them separated from day one.

This stack will serve you for the next decade because every component in it has a 10+ year track record in production, an active open-source community, and a clear upgrade path. The only proprietary dependency is Hasura, which has an open-source core and can be replaced with PostgREST or a hand-rolled GraphQL layer if needed. Everything else you own completely.